

C huffman.c X

codigos > C huffman.c > main

```
268
269 int main() {
270
271     int option = 0;
272     do {
273         printf("Bem vindo ao Algoritmo de Huffman.\nSelecione sua opcao:\n1 - Compactar\n2 - Descompactar\n");
274         scanf("%d", &option);
275         if (option == 1) {
276             char filename[256];
277             printf("Digite o nome do arquivo a ser compactado (com sua extensao). ");
278             scanf("%s", filename);
279             getchar();
280
```

C huffman.c X

codigos > C huffman.c > main

```
269 int main() {
280
281     char output[256];
282     strcpy(output, filename);
283     char *dot = strrchr(output, '.');
284     if (dot != NULL) *dot = '\0';
285     strcat(output, ".huff");
286
287     unsigned long long freqArr[256] = {0};
288     countFreq(filename, freqArr);
289
```

```
69 void countFreq(char *filename, unsigned long long *freqArr) {
70     FILE *file = fopen(filename, "rb");
71     if (file == NULL) {
72         printf("Erro ao abrir arquivo \"%s\"", filename);
73         return;
74     }
75
76     int currByte;
77     while ((currByte = fgetc(file)) != EOF) freqArr[currByte]++;
78     fclose(file);
79     return;
80 }
```

freqArr['B'] = 1; freqArr['A'] = 3; freqArr['N'] = 2

```
290 void *queue = fillQueue(freqArr);
```

```
82 void* fillQueue(unsigned long long *freqArr) {
83     void *qHead = NULL;
84
85     for (int i = 0; i < 256; i++) {
86         if (freqArr[i] > 0) {
87             void *node = createNode((unsigned char) i, freqArr[i]);
88             insertQueue(&qHead, node);
89         }
90     }
91
92     return qHead;
93 }
```

```
6 typedef struct Node {
7     unsigned char byte;
8     unsigned long long freq;
9     struct Node *left, *right;
10 }Node;
11
12 typedef struct NodeQ {
13     void *content;
14     struct NodeQ *next;
15 }NodeQ;
```

```
17 void* createNode(unsigned char byte, unsigned long long freq) {
18     Node *node = (Node*) malloc(sizeof(Node));
19     if (node == NULL) {
20         printf("Falha na alocação.\n");
21         return NULL;
22     }
23     node->byte = byte;
24     node->freq = freq;
25     node->left = NULL;
26     node->right = NULL;
27
28     return (void*)node;
29 }
```

```

52 void insertQueue(void *head, void *newNode) {
53     NodeQ **qHead = (NodeQ**) head; → atribuindo de void para nó de fila
54     NodeQ *node = (NodeQ*) createQueueElement(newNode); → ex. NodeQ 'N' NodeQ
55     if (node == NULL) return; → nó de árvore NULL
56     if (*qHead == NULL || compareNode((*qHead)->content, node->content) == 1) { → push ordenado
57         node->next = *qHead; } inserindo node na cabeça pela fila estar nula ou node ter menor frequência
58         *qHead = node;
59     } else {
60         NodeQ *aux = *qHead; → qHead → NULL → qHead → Node 'N' → NULL → qHead → Node 'B' → Node 'N' → NULL
61         while (aux->next != NULL && compareNode(node->content, aux->next->content) == 1) aux = aux->next; → avança até
62         node->next = aux->next; } insere ordenado Node 'A' freq(node) ≤ freq(aux->next)
63         aux->next = node; } avança até aux->next = NULL
64     }
65     return;
66 }

```

```

33 int compareNode(void* node1, void* node2) {
34     Node *a = (Node*) node1;
35     Node *b = (Node*) node2; } casts para nós de árvore
36
37     if (a->freq > b->freq) return 1; } flag para comparação de frequências
38     else return 0;
39 }
40
41 void* createQueueElement(void *content) { → "conversão" NodeQ
42     NodeQ *node = (NodeQ*) malloc(sizeof(NodeQ));
43     if (node == NULL) {
44         printf("Falha na alocação.\n");
45         return NULL;
46     }
47     node->content = content; } atribui nó de árvore como conteúdo do nó de fila
48     node->next = NULL;
49     return (void*) node;
50 }

```

```

290 void *queue = fillQueue(freqArr);

```



```

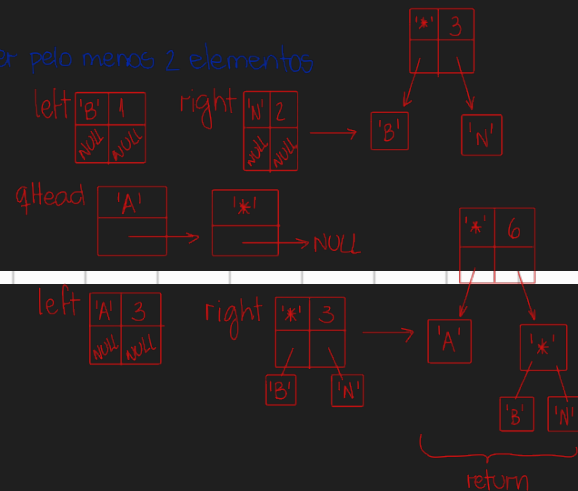
292 void *huffman = buildTree(&queue);

```

```

106 void* buildTree(void *head) {
107     NodeQ **qHead = (NodeQ**) head; → cast
108     while(*qHead != NULL && (*qHead)->next != NULL) { → enquanto a fila tiver pelo menos 2 elementos
109         Node *left = (Node*) popQ(qHead); → "byte"
110         Node *right = (Node*) popQ(qHead); → soma das freq
111         Node *parent = (Node*) createNode('*', left->freq + right->freq);
112         parent->left = left; } atribuição
113         parent->right = right;
114         insertQueue(qHead, parent); → insere ordenado
115     }
116     return popQ(qHead); → retorna o nó de árvore da cabeça da fila
117 }

```



```

95 void* popQ(void* head) {
96     NodeQ **qHead = (NodeQ**) head;
97     if (*qHead == NULL) return NULL;
98
99     NodeQ *aux = *qHead;
100     void* content = aux->content; → salva o nó de árvore
101     *qHead = (*qHead)->next; → avança a cabeça
102     free(aux);
103     return content; → devolve o nó de árvore
104 }

```

```

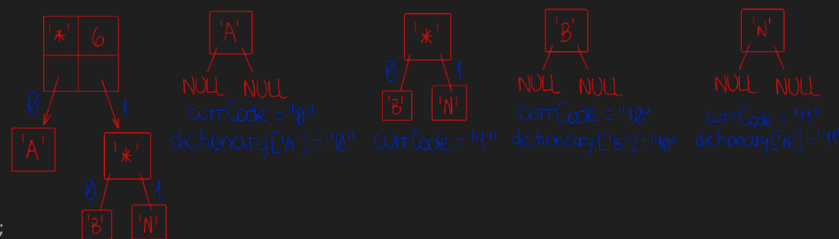
294 char *dictionary[256] = {NULL}; → array de strings para cada byte possível na tabela ASCII
295 char currCode[256]; → string temporária
296 codes(huffman, currCode, 0, dictionary);

```

```

130 void codes(void *treeRoot, char *currCode, int depth, char **dictionary) {
131     Node *root = (Node*) treeRoot; → cast
132     if (root == NULL) return; → para nó de árvore
133     if (root->left == NULL && root->right == NULL) {
134         currCode[depth] = '\0'; → índice para percorrer os chars da string
135         dictionary[root->byte] = strdup(currCode);
136         return; }
137     currCode[depth] = '0';
138     codes(root->left, currCode, depth+1, dictionary);
139     currCode[depth] = '1';
140     codes(root->right, currCode, depth+1, dictionary);
141 }

```



```

296 codes(huffman, currCode, 0, dictionary); dictionary['A'] = "0", dictionary['B'] = "10", dictionary['N'] = "11"
297
298 int sizeTree = treeSize(huffman);
299 int trashBits = trash(freqArr, dictionary);

```

```

119 int treeSize(void *treeRoot) { → retorna o número de bytes total da árvore para escrever no cabeçalho
120     Node *root = (Node*) treeRoot; → cast
121     if (root == NULL) return 0;
122
123     if (root->left == NULL && root->right == NULL) { → nó folha
124         if (root->byte == '*' || root->byte == '\\') return 2; → para que o código não se confunda com
125         else return 1; → nó folha comum os caracteres de escape, escrevemos \' ou \' se
126     } algum deles aparecer de forma literal no código,
127     return 1 + treeSize(root->left) + treeSize(root->right); o que dá dois bytes
128 }

```

* → return 1 + treeSize('A') + treeSize('*')
 A → return 1
 * → return 1 + treeSize('B') + treeSize('N')
 B → return 1
 N → return 1

} 5 bytes

```

161 int trash(unsigned long long *freqArr, char **dictionary) {
162     unsigned long long totalBits = 0;
163     for (int i = 0; i < 256; i++) {
164         if (freqArr[i] > 0) totalBits += freqArr[i] * strlen(dictionary[i]);
165     }
166     int resto = totalBits % 8; →
167     if (resto == 0) return 0;
168     else return 8 - resto; → 7 bits de lixo no último byte
169 }

```

"10"
 totalBitsB = 1 * 2 = 2
 totalBitsA = 3 * 1 = 3
 totalBitsN = 2 * 2 = 4

} 9 bits
menor múltiplo de 8 capaz de guardar = 16

```

301 FILE *outputFile = fopen(output, "wb"); → escreve byte a byte em arq huff
302 if (outputFile == NULL) {
303     printf("Erro ao criar arquivo para compactacao.\n");
304     return 1;
305 }
306 writeHeader(outputFile, trashBits, sizeTree, huffman);
307 writeCompressed(filename, outputFile, dictionary);
308
309 printf("Arquivo compactado em %s\n", output);
310 fclose(outputFile);

```

```

171 void writeHeader(FILE *output, int trash, int treeSize, void *treeRoot) {
172     //faça na mão usando trash = 5 e treeSize = 1209
173     unsigned char byte1 = (trash << 5) | (treeSize >> 8); → trash 00000111 treeSize 00000101 → 11100000
174     unsigned char byte2 = treeSize & 0xFF; → 00000101 & 11111111 → 1 00000000
175     fputc(byte1, output); escrevemos & 11111111
176     fputc(byte2, output); lixo e tamanho 00000101
177     writeTree(treeRoot, output); da árvore no cabeçalho
178 }

```

```

147 void writeTree(void *treeRoot, FILE *file) {
148     Node *root = (Node*) treeRoot;
149     if (root == NULL) return;
150
151     if (root->left == NULL && root->right == NULL) { → nó folha
152         if (root->byte == '*' || root->byte == '\\') fputc('\\', file); → proteção se '*' ou '\' estiverem
153         fputc(root->byte, file); → escrita do byte em si literalmente no arquivo
154     } else {
155         fputc('*', file); → nó pa
156         writeTree(root->left, file); } escrita em pré-ordem
157         writeTree(root->right, file);
158     }
159 }

```

11100000 00000101 *A*BN

```

180 void writeCompressed(char *filename, FILE *output, char **dictionary) {
181     FILE *input = fopen(filename, "rb"); → abre o arquivo original para ler byte a byte e fazer a tradução
182     if (input == NULL) {
183         printf("Erro ao abrir arquivo %s\n", filename);
184         return;
185     }
186
187     unsigned char buffer = 0;
188     int bitCounter = 0;
189     int currByte;
190     while ((currByte = fgetc(input)) != EOF) {
191         char *code = dictionary[currByte]; → acessa os bits traduzidos de cada byte
192         for (int i = 0; code[i] != '\0'; i++) {
193             buffer = buffer << 1;
194             if (code[i] == '1') buffer = buffer | 1;
195             bitCounter++;
196
197             if (bitCounter == 8) {
198                 fputc(buffer, output);
199                 buffer = 0;
200                 bitCounter = 0;
201             }
202         }
203     }
204     //colocando o lixo na frente do cabeçalho
205     if (bitCounter > 0) {
206         buffer = buffer << (8 - bitCounter);
207         fputc(buffer, output);
208     }
209     fclose(input);
210 }

```

} assim que completamos 8 bits no buffer atual, escrevemos o byte e zeramos os contadores

'B': buffer = 0 ; bitCounter = 0
code[0] = 1 | buffer = 1 ; bitCounter = 1
code[1] = 0 | buffer = 10 ; bitCounter = 2

'A':
code[0] = 0 | buffer = 100 ; bitCounter = 3

'N':
code[0] = 1 | buffer = 1001 ; bitCounter = 4
code[1] = 1 | buffer = 10011 ; bitCounter = 5

'A':
code[0] = 0 | buffer = 100110 ; bitCounter = 6

'N':
code[0] = 1 | buffer = 1001101 ; bitCounter = 7
code[1] = 1 | buffer = 10011011 ; bitCounter = 8
→ escreve byte no arquivo e zera contador

'A':
code[0] = 0 | buffer = 0 ; bitCounter = 1
→ buffer > 0?
buffer = buffer << (8 - bitCounter) → abre espaço para o lixo
buffer = 0 << 7 = 00000000 → escreve byte
lixo

- Tamanho Inicial: 6 bytes (1 por letra)
- Tamanho Final: 9 bytes (2 de cabeçalho, 5 de árvore, 2 de dados)

Usaremos esse novo arquivo na descompressão

```
311 } else if (option == 2) {
312     char filename[256];
313     printf("Digite o nome do arquivo a ser descompactado (com sua extensao). ");
314     scanf("%s", filename); → arq.huff
315     getchar();
316
317     char ext[50];
318     printf("Digite a extensao para o qual deseja descompactar (ex: pdf, png, jpeg...). ");
319     scanf("%s", ext); → txt
320     getchar();
321
322     char output[256];
323     strcpy(output, filename);
324     char *dot = strrchr(output, '.');
325     if(dot != NULL) *dot = '\0'; → output = "arq\0"
326     strcat(output, "."); } output = "arq.txt"
327     strcat(output, ext);
328
329     FILE *input = fopen(filename, "rb");
330     int byte1 = fgetc(input); } capturamos os dois primeiros bytes do arquivo comprimido
331     int byte2 = fgetc(input);
```

```
333 //descobrimos o tamanho do lixo (3 primeiros bits)
334 int trashSize = byte1 >> 5; → isolamos os 3 primeiros bits : trashSize = 11100000 >> 5 = 00000111 = 7
335 //montamos o tamanho da arvore isolando os 5 bits restantes do primeiro byte com 0x1F (0001 1111) e deslocamos 8 posições
336 //para abrir espaço para o segundo byte, juntando com um OR
337 int treeSize = ((byte1 & 0x1F) << 8) | byte2; → byte1 11100000 1 byte 2 00000101 } 00000101 = 5
338 int index = 0; 0x1F 1 00011111 } 00000000 << 8 = 00000000 00000000 } treeSize
339 void *huffmanTree = rebuildTree(treeSize, &index, input);
```

```
213 void* rebuildTree(int treeSize, int *index, FILE *input) {
214     if ((*index) >= treeSize) return NULL; → funciona como um for para percorrer a função treeSize vezes
215
216     int currByte = fgetc(input); → pegamos o terceiro byte *A*BN
217     (*index)++; *A*BN = 5 bytes
218
219     if (currByte == '\\') { → caractere de escape: próx nó é uma folha
220         //fgetc retorna um int, logo não podemos criar currByte como char direto
221         int currByte = fgetc(input); → avançamos ao próx byte após aviso (dado de fato)
222         (*index)++; → avançamos o índice
223         return createNode((unsigned char)currByte, 0); sempre '\ ' ou '*'.
224     } else if (currByte == '*') { ← ← ←
225         Node *node = (Node*) createNode('*', 0); → freq. não importa
226
227         //pré-ordem
228         node->left = (Node*) rebuildTree(treeSize, index, input); A B
229         node->right = (Node*) rebuildTree(treeSize, index, input); * N
230
231         return (void*) node;
232     } else { ← ← ←
233         //nó folha normal (caractere aleatório)
234         return createNode((unsigned char)currByte, 0);
235     }
236 }
```

A nível de curiosidade, se alterarmos o byte após '\ ' para qualquer outro que não seja '*' ou '\ ' o código não quebra. Ele apenas segue acreditando que o byte é um nó.

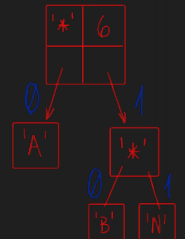
```
340 FILE *outputFile = fopen(output, "wb"); → cria arq.txt (ou sobrescreve)
341 if (outputFile == NULL) {
342     printf("Erro ao criar arquivo para descompactar.\n");
343     fclose(input);
344     return 1;
345 }
346
347 writeDescompressed(input, outputFile, huffmanTree, trashSize);
348 printf("Arquivo descompactado em %s\n", output);
349 fclose(input);
350 fclose(outputFile);
351 }
352 } while (option != 1 && option != 2); → trava
353
354 return 0;
355 }
```

```

238 void writeDecompressed(FILE *input, FILE *output, void *treeRoot, int trash) {
239     Node *root = (Node*) treeRoot; → conversão
240     Node *currNode = root; → auxiliar para navegação
241
242     int currByte = fgetc(input); 10011011
243     int nextByte;
244
245     while (currByte != EOF) {
246         nextByte = fgetc(input); 00000000
247
248         //se o próximo byte for o EOF, descontamos o lixo do byte atual, pois chegamos no ultimo
249         int bits = (nextByte == EOF) ? (8 - trash) : 8; → checagem para último byte
250         bits = 8
251         //lemos da esquerda para a direita até o eventual primeiro bit do lixo
252         for (int i = 7; i >= 8 - bits; i--) { → leitura bit a bit
253             //isolando o bit atual
254             int currBit = (currByte >> i) & 1; → 10011011 >> 7 = 00000001 & 00000001 = 00000001
255             if (currBit) currNode = currNode->right; → se bit = 1, avançamos pela direita → *
256             else currNode = currNode->left; → se não, esquerda
257
258             if (currNode->left == NULL && currNode->right == NULL) {
259                 //se chegar a um nó folha, recuperamos o byte e escrevemos no arquivo de saída
260                 fputc(currNode->byte, output);
261                 currNode = root; //voltando à raiz para o próximo byte
262             }
263         }
264
265         currByte = nextByte; //avançamos o byte atual
266     }
267 }

```

huffmanTree =



Em arg.txt teremos "BANANA"